

Problem Set #1

Emorie D Beck

1/9/26

Part 1 (Week 1)

Welcome to Data Cleaning and Management using R! This problem set is intended to give you some practice becoming familiar with using R. In this problem set, I'm asking you to: create an R project; render to pdf; load and investigate an R data frame that is stored on the web; and apply some basic functions to atomic vectors.

- Note: Change the values of the YAML header above to your name and the date.

```
library(tidyverse)
```

Warning: package 'purrr' was built under R version 4.4.3

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
v dplyr      1.1.4      v readr      2.1.5
v forcats    1.0.0      v stringr    1.6.0
v ggplot2    3.5.2      v tibble     3.2.1
v lubridate  1.9.4      v tidyr      1.3.1
v purrr      1.2.1
-- Conflicts ----- tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()     masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become
```

Question 1: Creating an R project

Create an R project

- Create a folder where you want to save files associated with problem set 1. Let's call that folder "problemset1", but you can name it whatever you want.
 - For instance, it could be psc290-fq23 » problem-sets » problemset1.
- In RStudio, click on "File" » "New Project" » "Existing Directory" » "Browse".
- Browse to find and select your problem set 1 folder.
- Click on "Create Project".
 - An R project file has the extension ".Rproj".
 - The name of the file should be "problemset1.Rproj", or whatever you named the folder.

Save this problemset1.qmd file anywhere in the folder named problemset1.

- Use this naming convention "lastname-firstname-ps#" for your .qmd files (e.g. beck__emorie__ps1.qmd).
 - If you want, you can change the name of this file to include your first and last name.
- Run the `getwd()` function and the `list.files()` function in the code chunk below.
- What is the output? Why?

```
getwd() # list current directory
list.files() # list files in current directory
# mine may be different from yours because I have the data
```

ANSWER: `getwd()` lists the current directory path, which is important for relative file paths. `list.files()` lists all the files in current working directory (listed in `getwd()`).

Question 2: Render to pdf

- At the top of this .qmd file, type in your first and last name in the appropriate place in the YAML header (e.g. "Hadley Wickham").
- In the date field of the YAML header, insert the date within quotations (any date format is fine).
- Now click the "Render" button near the top of your RStudio window (icon with blue yarn ball).
 - Alternatively you can use the shortcut: **Cmd/Ctrl + Shift + k**.
 - *Note:* One goal of this assignment is to make sure you are able to render to a PDF without running into errors.

Question 3: Load .Rdata directly with url and then investigate the data frame

1. This question asks you to load a dataframe by specifying the `url()` function within the `load()` function.
 - Url link for data frame: <https://github.com/emoriebeck/psc290-data-FQ23/raw/main/05-assignments/01-ps1/pwe-ps1-small.RData>
 - Hint: to load .Rdata use the `load()` and `url()` functions because you are using a link. follow this approach: `load(url("url_link"))`.
 - * Note: the `url_link` is put within quotes

Load the dataframe within this code chunk below.

```
#?load
load(url("https://github.com/emoriebeck/psc290-data-FQ23/raw/main/05-assignments/01-ps1/pwe-ps1-small.RData"))
```

2. Print the data frame `df_pwe` by typing its name.

```
df_pwe
```

```
# A tibble: 100 x 45
  country introelapse testelapse surveyelapse TIPI1 TIPI2 TIPI3 TIPI4 TIPI5
  <chr>      <dbl>      <dbl>      <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1 US          1118          103          111     2     6     1     7     7
2 GB              8          116          128     3     2     6     1     5
3 US              9          142          217     5     6     7     5     5
4 US              2           79           90     2     5     1     2     3
5 VN          545          210          276     3     1     6     5     5
6 IN         4397          625           79     4     5     2     6     7
7 NO              2          181          136     3     5     3     6     7
8 NONE          4          208          198     5     6     2     5     7
9 PL             13          191          266     6     7     4     3     5
10 IN            37          394          380     7     1     7     1     1
# i 90 more rows
# i 36 more variables: TIPI6 <dbl>, TIPI7 <dbl>, TIPI8 <dbl>, TIPI9 <dbl>,
# TIPI10 <dbl>, VCL1 <dbl>, VCL2 <dbl>, VCL3 <dbl>, VCL4 <dbl>, VCL5 <dbl>,
# VCL6 <dbl>, VCL7 <dbl>, VCL8 <dbl>, VCL9 <dbl>, VCL10 <dbl>, VCL11 <dbl>,
# VCL12 <dbl>, VCL13 <dbl>, VCL14 <dbl>, VCL15 <dbl>, VCL16 <dbl>,
# education <dbl>, urban <dbl>, gender <dbl>, engnat <dbl>, age <dbl>,
# screenw <dbl>, screenh <dbl>, hand <dbl>, religion <dbl>, ...
```

3. Use the `typeof()` function to investigate the type of data frame `df_pwe`.

```
typeof(df_pwe) # tibble
```

```
[1] "list"
```

4. Apply the `length()` function to the data frame `df_pwe`. What does this output mean in your own words?

```
length(df_pwe)
```

```
[1] 45
```

ANSWER: `length()` captures the number of columns in the data frame. It's better practice to use `ncol()` and `nrow()`, which have more interpretable behavior.

5. Use the `str()` function to investigate the structure of the data frame `df_pwe`.

```
str(df_pwe)
```

```
tibble [100 x 45] (S3: tbl_df/tbl/data.frame)
 $ country      : chr [1:100] "US" "GB" "US" "US" ...
 $ introelapse  : num [1:100] 1118 8 9 2 545 ...
 $ testelapse   : num [1:100] 103 116 142 79 210 625 181 208 191 394 ...
 $ surveyelapse: num [1:100] 111 128 217 90 276 79 136 198 266 380 ...
 $ TIPI1        : num [1:100] 2 3 5 2 3 4 3 5 6 7 ...
 $ TIPI2        : num [1:100] 6 2 6 5 1 5 5 6 7 1 ...
 $ TIPI3        : num [1:100] 1 6 7 1 6 2 3 2 4 7 ...
 $ TIPI4        : num [1:100] 7 1 5 2 5 6 6 5 3 1 ...
 $ TIPI5        : num [1:100] 7 5 5 3 5 7 7 7 5 1 ...
 $ TIPI6        : num [1:100] 7 6 5 5 6 4 5 3 1 6 ...
 $ TIPI7        : num [1:100] 7 5 1 1 7 5 5 7 5 7 ...
 $ TIPI8        : num [1:100] 7 2 1 7 1 6 5 1 6 1 ...
 $ TIPI9        : num [1:100] 1 6 6 5 6 3 5 6 6 7 ...
 $ TIPI10       : num [1:100] 1 5 7 3 3 2 1 2 4 4 ...
 $ VCL1         : num [1:100] 1 1 1 1 1 1 1 0 1 0 ...
 $ VCL2         : num [1:100] 1 1 1 1 1 0 1 1 0 0 ...
 $ VCL3         : num [1:100] 0 1 1 0 0 0 0 0 0 0 ...
 $ VCL4         : num [1:100] 1 1 1 0 1 1 1 1 1 0 ...
 $ VCL5         : num [1:100] 1 1 1 1 1 1 1 0 1 1 ...
 $ VCL6         : num [1:100] 0 0 0 0 0 0 0 1 0 0 ...
 $ VCL7         : num [1:100] 0 1 0 0 1 0 0 0 0 0 ...
```

```

$ VCL8      : num [1:100] 0 1 0 0 0 1 1 1 1 0 ...
$ VCL9      : num [1:100] 0 0 0 0 0 0 0 0 0 0 ...
$ VCL10     : num [1:100] 1 1 1 0 1 1 1 1 1 1 ...
$ VCL11     : num [1:100] 0 1 0 1 1 0 1 1 1 0 ...
$ VCL12     : num [1:100] 0 0 0 0 0 0 0 0 0 0 ...
$ VCL13     : num [1:100] 1 1 1 1 0 1 1 0 0 0 ...
$ VCL14     : num [1:100] 1 1 1 0 1 1 1 1 0 0 ...
$ VCL15     : num [1:100] 1 1 1 1 1 1 1 1 1 0 ...
$ VCL16     : num [1:100] 1 1 1 1 1 1 1 1 1 1 ...
$ education : num [1:100] 2 3 2 2 2 3 1 2 2 4 ...
$ urban     : num [1:100] 1 1 2 2 3 3 2 3 3 1 ...
$ gender    : num [1:100] 2 2 2 1 2 1 2 2 2 1 ...
$ engnat    : num [1:100] 1 1 1 1 2 2 2 2 2 2 ...
$ age       : num [1:100] 19 64 18 25 19 22 32 21 19 26 ...
$ screenw   : num [1:100] 1093 768 1366 1536 1152 ...
$ screenh   : num [1:100] 615 1024 768 864 720 ...
$ hand      : num [1:100] 1 1 1 1 1 1 1 1 1 1 ...
$ religion   : num [1:100] 1 2 2 1 3 8 1 2 2 8 ...
$ orientation : num [1:100] 1 1 1 1 1 1 1 1 5 2 ...
$ race      : num [1:100] 16 16 16 13 11 11 16 16 16 11 ...
$ voted     : num [1:100] 2 2 1 1 2 1 2 1 1 1 ...
$ married   : num [1:100] 1 3 1 1 1 1 1 1 1 2 ...
$ familysize : num [1:100] 2 2 3 3 2 2 2 2 1 3 ...
$ major     : chr [1:100] NA "Psychology" "Chemistry" NA ...

```

6. Use the `names` function to list the names of the elements (variables) within `df_pwe`.

```
names(df_pwe)
```

```

[1] "country"      "introelapse"  "testelapse"   "surveyelapse" "TIPI1"
[6] "TIPI2"        "TIPI3"        "TIPI4"        "TIPI5"        "TIPI6"
[11] "TIPI7"        "TIPI8"        "TIPI9"        "TIPI10"       "VCL1"
[16] "VCL2"        "VCL3"        "VCL4"        "VCL5"        "VCL6"
[21] "VCL7"        "VCL8"        "VCL9"        "VCL10"       "VCL11"
[26] "VCL12"       "VCL13"       "VCL14"       "VCL15"       "VCL16"
[31] "education"    "urban"        "gender"       "engnat"       "age"
[36] "screenw"     "screenh"     "hand"        "religion"     "orientation"
[41] "race"        "voted"       "married"     "familysize"   "major"

```

7. Wrap your answer above — `names(data_frame_name)` — within the `typeof()` function. Do the same for the `length()` function, and the `str()` function as well. Interpret what the output means in your own words.

```
typeof(names(df_pwe))
```

```
[1] "character"
```

```
length(names(df_pwe))
```

```
[1] 45
```

ANSWER: `names()` provides a character vector listing the column names of `df_pwe`. Thus, `typeof(names())` will always tell you it's a character vector, and `length(names())` will always provide an integer with the number of columns in the data frame.

Question 4: Applying basic functions to atomic vectors

1. Create an atomic vector object named `age` with the following values: 3, 6, 41, 43.

```
age <- c(3, 6, 41, 43)
```

2. Apply the `typeof()`, `length()`, and `str()` functions to the object `age`.

```
typeof(age) # double
```

```
[1] "double"
```

```
length(age) # 4
```

```
[1] 4
```

```
str(age)      # num [1:4] 3 6 41 43
```

```
num [1:4] 3 6 41 43
```

3. Apply the `sum()` function to `age`.

```
sum(age)
```

```
[1] 93
```

4. Apply the `sum()` function to `age` but this time include the argument `na.rm = FALSE`.

```
sum(age, na.rm = F)
```

```
[1] 93
```

5. In general, what is a function “argument name” and what is an “argument value”? What does the argument `na.rm` do?

ANSWER: Argument names are generally the names of function-specific arguments. Argument values are the input to those function-specific arguments. `?function` (e.g., `?sum`) will help you understand what arguments a function takes and what accurate argument values will be.

6. Create a new object `age2` with the following values: 3, 6, 41, 43, NA. Now calculate the sum of `age2` using the argument `na.rm = FALSE` and then calculate the sum using the argument `na.rm = TRUE`. Explain why the outputs of these two `sum()` functions differ.

```
age2 <- c(3, 6, 41, 43, NA)
sum(age2, na.rm = F)
```

```
[1] NA
```

```
sum(age2, na.rm = T)
```

```
[1] 93
```

ANSWER: Now that there is a missing (NA) value, any sums, means, etc. will return as NA unless `na.rm = T`. When `na.rm = T`, the missing values are removed (hence the `.rm`).

7. Create a vector `tf` using the following code: `tf <- c(TRUE,FALSE,TRUE,FALSE,TRUE)`. Next apply the `typeof()`, `length()`, and `str()` functions to the object `tf`. Based on this output, briefly describe the object `tf` in your own words (one sentence is fine).

```
tf <- c(TRUE,FALSE,TRUE,FALSE,TRUE)
typeof(tf)
```

```
[1] "logical"
```

```
length(tf)
```

```
[1] 5
```

```
str(tf)
```

```
logi [1:5] TRUE FALSE TRUE FALSE TRUE
```

ANSWER: `tf` is a logical vector containing `TRUE`'s and `FALSE`s.

8. Apply the `sum()` function to the object, using the option to remove `NA` values prior to calculation. What numeric value do mathematical calculations in R assign to `TRUE` values and what do they assign to `FALSE` values?

```
sum(tf, na.rm = T)
```

```
[1] 3
```

ANSWER: `TRUE`s are assigned 1's and `FALSE`s are 0's. So the result here indicates that there are 3 trues in the vector.

9. This is the syntax of the `mean()` function that includes both argument names and the default values for arguments: `mean(x, trim = 0, na.rm = FALSE)`.

When using a function, R requires you to type the values you assign to each argument, but typing in the argument names is usually optional. Even though it takes a bit more time, I usually like typing in both argument names and argument values, because it forces me to be more conscious about what value I am assigning to which argument, especially when a function is new to me.

Use the `mean()` function to calculate the mean of object `tf` (removing `NA` values prior to calculation). In your function call, include both the argument name and the argument value for each argument (argument value for the `trim` argument can be 0). Then run the same function, but without typing any argument names.

```
mean(tf, trim = 0, na.rm = T)
```

```
[1] 0.6
```



```
mean(tf, 0, T)
```

```
[1] 0.6
```

Part 2 (Week 2)

The aim of part 2 is to give you practice completing data management tasks associated with filtering/isolating observations, sorting observations, and selecting variables. This can be done using the `filter()`, `arrange()`, and `select()` functions from the `tidyverse` package. Filtering/sorting the data can also be done using `base R`'s subsetting operators and `subset()/order()` functions (not covered in class but examples provided below).

For the following questions, you'll be asked to complete the same task multiple ways based on the `tidyverse` and `base R` approaches. We want you to understand that there are several ways to complete the same task and we want you to practice completing the same task in different ways.

Question 1: Load and inspect `df_event` dataset

1. In the code chunk below, complete the following:

- Load the `tidyverse` library
- Use the `load()` and `url()` functions to download the `df_event` dataframe from the url: https://github.com/emoriebeck/psc290-data-FQ23/raw/main/05-assignments/02-ps2/ps2_data/df_event.csv
 - Each row in `df_event` represents a recruiting visit

```
rm(list = ls())
library(tidyverse)
load(url("https://github.com/emoriebeck/psc290-data-FQ23/raw/main/05-assignments/02-ps2/ps2_data/df_event.csv"))
df_event
```

```
# A tibble: 18,680 x 32
```

	instnm	univ_id	instst	pid	event_date	event_type	zip	school_id	ipeds_id
	<chr>	<int>	<chr>	<int>	<date>	<chr>	<chr>	<chr>	<int>
1	UM Amher~	166629	MA	57570	2017-10-12	public hs	01002	25019200~	NA
2	UM Amher~	166629	MA	56984	2017-10-04	public hs	01007	25024300~	NA
3	UM Amher~	166629	MA	57105	2017-10-25	public hs	01020	25036600~	NA
4	UM Amher~	166629	MA	57118	2017-10-26	public hs	01020	25036600~	NA
5	Stony Br~	196097	NY	16281	2017-10-02	public hs	01027	25045900~	NA

```

6 USCC      218663 SC      8608 2017-09-18 private hs 01027 00604341      NA
7 UM Amher~ 166629 MA      56898 2017-09-18 private hs 01027 00604341      NA
8 UM Amher~ 166629 MA      56933 2017-09-26 public hs 01033 25054000~    NA
9 UM Amher~ 166629 MA      56940 2017-09-26 private hs 01033 00603949      NA
10 UM Amher~ 166629 MA      57030 2017-10-12 public hs 01038 25059400~    NA

```

```
# i 18,670 more rows
```

```

# i 23 more variables: event_state <chr>, med_inc <dbl>, pop_total <dbl>,
#   pct_white_zip <dbl>, pct_black_zip <dbl>, pct_asian_zip <dbl>,
#   pct_hispanic_zip <dbl>, pct_amerindian_zip <dbl>,
#   pct_nativehawaii_zip <dbl>, pct_tworaces_zip <dbl>,
#   pct_otherrace_zip <dbl>, fr_lunch <dbl>, titlei_status_pub <fct>,
#   total_12 <dbl>, school_type_pri <int>, school_type_pub <int>, ...

```

2. Inspect the `df_event` dataframe:

- Use `names()` to identify the column names in the dataframe
- Use `typeof()` to show the data type of the `event_state` column
- Use `str()` to show the structure of the `med_inc` column
- Use `table()` to show the categorical values of the `event_type` column

```
names(df_event) # column names
```

```

[1] "instnm"          "univ_id"          "instst"
[4] "pid"             "event_date"        "event_type"
[7] "zip"             "school_id"         "ipeds_id"
[10] "event_state"     "med_inc"           "pop_total"
[13] "pct_white_zip"   "pct_black_zip"     "pct_asian_zip"
[16] "pct_hispanic_zip" "pct_amerindian_zip" "pct_nativehawaii_zip"
[19] "pct_tworaces_zip" "pct_otherrace_zip" "fr_lunch"
[22] "titlei_status_pub" "total_12"          "school_type_pri"
[25] "school_type_pub" "g12offered"        "g12"
[28] "total_students_pub" "total_students_pri" "event_name"
[31] "event_location_name" "event_datetime_start"

```

```
typeof(df_event$event_state) # "vector" type of event_state column
```

```
[1] "character"
```

```
str(df_event$med_inc) # structure of the med_inc column
```

```
num [1:18680] 71714 89122 70136 70136 71024 ...
```

```
# hint because it's a column grabbed using the $, it's just a vector and its structure
# is just values

table(df_event$event_type) # counts of where events took place
```

2yr college	4yr college	other	private hs	public hs
951	531	2001	3774	11423

Question 2: Filtering/isolating observations

Filtering can be done using multiple approaches: `tidyverse`'s `filter()` function, base R's subsetting operators, and base R's `subset()` function. Here is an example of using each method to obtain the total number of recruiting visits to California from the `df_event` dataframe:

```
# tidyverse using filter()
nrow(filter(df_event, event_state == 'CA'))

# base R using subsetting operators
nrow(df_event[df_event$event_state == 'CA', ])

# base R using subset()
nrow(subset(df_event, event_state == 'CA'))
```

1. Your turn! Count the number of recruiting events that satisfy all the following criteria:

- By the University of Massachusetts-Amherst (`univ_id`: 166629)
- An out-of-state public high school (use `event_type`, `event_state`, and `instst`, which is the visiting university's home state)
- Average median household income is greater than or equal to \$100,000 (`med_inc`)
- Make sure to drop any NA values

Use `nrow()` to obtain the count. Do the filtering in the 3 ways below. You should get the same answer.

tidyverse using `filter()`:

```
cols <- c("univ_id", "event_type", "event_state", "instst", "med_inc")

df_event %>%
```

```

filter(
  complete.cases(df_event %>% select(all_of(cols))) &
  univ_id == 166629 & # umass amherst by ID
  event_type == "public hs" & # public only
  event_state != "MA" & # event not in MA
  instst == "MA" & # but the institution is [because it's umass amherst]
  med_inc >= 100000 # median household income >= 100,000
) %>%
nrow()

```

[1] 264

base R using subsetting operators (hint: use `which()` to drop NAs):

```

nrow(
  with(df_event, df_event[
    unique(which(!is.na(df_event[,cols]), arr.ind = T)[,"row"]) &
    univ_id == 166629 & # umass amherst by ID
    event_type == "public hs" & # public only
    event_state != "MA" & # event not in MA
    instst == "MA" & # but the institution is [because it's umass amherst]
    med_inc >= 100000 # median household income >= 100,000,]
  ,])
)

```

[1] 266

base R using `subset()`:

```

nrow(
  subset(
    df_event,
    subset = complete.cases(subset(df_event, select = cols)) &
    univ_id == 166629 & # umass amherst by ID
    event_type == "public hs" & # public only
    event_state != "MA" & # event not in MA
    instst == "MA" & # but the institution is [because it's umass amherst]
    med_inc >= 100000 # median household income >= 100,000
  )
)

```

[1] 264

2. Count the number of recruiting events that satisfy all the following criteria:

- By the University of South Carolina-Columbia (`univ_id`: 218663) or by the University of Alabama (`univ_id`: 100751)
- And either:
 - An in-state 2-year college visit (use `event_type`, `event_state`, and `instst`, which is the visiting university's home state) OR
 - A zip code with population under 10,000 (use `pop_total`)
- Make sure to drop any NA values
- Note the [order of precedence](#): `&` is higher in priority than `|`

tidyverse using `filter()`:

```
cols <- c("univ_id", "event_type", "event_state", "instst", "pop_total")

df_event %>%
  filter(
    complete.cases(df_event %>% select(all_of(cols))) &
    univ_id %in% c(218663, 100751) &
    ((event_type == "2yr college" & event_state == instst) |
     pop_total < 10000)
  ) %>%
  nrow()
```

[1] 524

base R using subsetting operators (hint: use `which()` to drop NAs):

```
cols <- with(df_event, !is.na(univ_id) & !is.na(event_type) & !is.na(event_state) & !is.na(
nrow(
  with(df_event, df_event[
    which(cols) &
    univ_id %in% c(218663, 100751) &
    ((event_type == "2yr college" & event_state == instst) |
     pop_total < 10000)
  , ]))
)
```

Warning in which(cols) & univ_id %in% c(218663, 100751): longer object length is not a multiple of shorter object length

[1] 622

base R using subset():

```
cols <- c("univ_id", "event_type", "event_state", "instst", "pop_total")

nrow(
  subset(
    df_event,
    subset = complete.cases(subset(df_event, select = cols)) &
      complete.cases(df_event %>% select(all_of(cols))) &
    univ_id %in% c(218663, 100751) &
    ((event_type == "2yr college" & event_state == instst) |
     pop_total < 10000)
  )
)
```

[1] 524

Question 3: Sorting observations

1. Create a new dataframe that contains the events in `df_events` sorted by:

- Ascending `univ_id`
- Ascending `event_date`
- Ascending `event_state`
- Descending `pct_white_zip`
- Descending `med_inc`

Then preview the first 10 rows using `head()`. Do this in 2 ways: using `tidyverse`'s `arrange()` and base R's `order()`.

`tidyverse` using `arrange()`:

```
df_event %>%
  arrange(univ_id, event_date, event_state, desc(pct_white_zip), desc(med_inc))
```

```
# A tibble: 18,680 x 32
  instnm univ_id instst  pid event_date event_type zip school_id ipeds_id
  <chr>   <int> <chr>  <int> <date>      <chr>      <chr> <chr>      <int>
1 Bama    100751 AL      2667 2017-01-10 private hs  75001 X1328481      NA
2 Bama    100751 AL      2674 2017-01-11 2yr college 35010 <NA>      100760
3 Bama    100751 AL      2675 2017-01-11 other       35044 <NA>      NA
4 Bama    100751 AL      2691 2017-01-12 private hs  75244 A0303150      NA
5 Bama    100751 AL      2676 2017-01-17 2yr college 36350 <NA>      101286
6 Bama    100751 AL      2851 2017-01-17 public hs   21769 2400330006~    NA
7 Bama    100751 AL      2733 2017-01-17 public hs   75002 4807890001~    NA
8 Bama    100751 AL      2677 2017-01-18 2yr college 36330 <NA>      101143
9 Bama    100751 AL      2645 2017-01-18 public hs   30277 1301500020~    NA
10 Bama    100751 AL      2736 2017-01-18 public hs   30281 1302820012~    NA
# i 18,670 more rows
# i 23 more variables: event_state <chr>, med_inc <dbl>, pop_total <dbl>,
#   pct_white_zip <dbl>, pct_black_zip <dbl>, pct_asian_zip <dbl>,
#   pct_hispanic_zip <dbl>, pct_amerindian_zip <dbl>,
#   pct_nativehawaii_zip <dbl>, pct_tworaces_zip <dbl>,
#   pct_otherrace_zip <dbl>, fr_lunch <dbl>, titlei_status_pub <fct>,
#   total_12 <dbl>, school_type_pri <int>, school_type_pub <int>, ...
```

base R using order():

```
df_event[with(df_event, order(univ_id, event_date, event_state, desc(pct_white_zip), desc(
```

```
# A tibble: 18,680 x 32
  instnm univ_id instst  pid event_date event_type zip school_id ipeds_id
  <chr>   <int> <chr>  <int> <date>      <chr>      <chr> <chr>      <int>
1 Bama    100751 AL      2667 2017-01-10 private hs  75001 X1328481      NA
2 Bama    100751 AL      2674 2017-01-11 2yr college 35010 <NA>      100760
3 Bama    100751 AL      2675 2017-01-11 other       35044 <NA>      NA
4 Bama    100751 AL      2691 2017-01-12 private hs  75244 A0303150      NA
5 Bama    100751 AL      2676 2017-01-17 2yr college 36350 <NA>      101286
6 Bama    100751 AL      2851 2017-01-17 public hs   21769 2400330006~    NA
7 Bama    100751 AL      2733 2017-01-17 public hs   75002 4807890001~    NA
8 Bama    100751 AL      2677 2017-01-18 2yr college 36330 <NA>      101143
9 Bama    100751 AL      2645 2017-01-18 public hs   30277 1301500020~    NA
10 Bama    100751 AL      2736 2017-01-18 public hs   30281 1302820012~    NA
# i 18,670 more rows
# i 23 more variables: event_state <chr>, med_inc <dbl>, pop_total <dbl>,
#   pct_white_zip <dbl>, pct_black_zip <dbl>, pct_asian_zip <dbl>,
#   pct_hispanic_zip <dbl>, pct_amerindian_zip <dbl>,
```

```
# pct_nativehawaii_zip <dbl>, pct_tworaces_zip <dbl>,
# pct_otherrace_zip <dbl>, fr_lunch <dbl>, titlei_status_pub <fct>,
# total_12 <dbl>, school_type_pri <int>, school_type_pub <int>, ...
```

Question 4: Selecting variables

1. Create a new dataframe by selecting the columns `univ_id`, `event_date`, `event_type`, `zip`, and `med_inc` from `df_event`. Use the `names()` function to show what columns (variables) are in the newly created dataframe.

Do this in 3 ways: using tidyverse's `select()`, base R's subsetting operators, and base R's `subset()`.

tidyverse using `select()`:

```
df_event %>%
  select(univ_id, event_date, event_type, zip, med_inc) %>%
  names()
```

```
[1] "univ_id"      "event_date" "event_type" "zip"          "med_inc"
```

base R using subsetting operators:

```
names(df_event[, c("univ_id", "event_date", "event_type", "zip", "med_inc")])
```

```
[1] "univ_id"      "event_date" "event_type" "zip"          "med_inc"
```

base R using `subset()`:

```
subset(
  df_event,
  select = c("univ_id", "event_date", "event_type", "zip", "med_inc")
)
```

```
# A tibble: 18,680 x 5
  univ_id event_date event_type zip    med_inc
  <int> <date>      <chr>   <chr>   <dbl>
1  166629 2017-10-12 public hs   01002  71714.
2  166629 2017-10-04 public hs   01007  89122.
3  166629 2017-10-25 public hs   01020  70136.
```



```

4 166629 2017-10-26 public hs 01020 70136.
5 196097 2017-10-02 public hs 01027 71024.
6 218663 2017-09-18 private hs 01027 71024.
7 166629 2017-09-18 private hs 01027 71024.
8 166629 2017-09-26 public hs 01033 97225
9 166629 2017-09-26 private hs 01033 97225
10 166629 2017-10-12 public hs 01038 77800.
# i 18,670 more rows

```

Question 5: Additional practice with df_school_all dataframe

1. In the code chunk below, complete the following:
 - Use the `load()` and `url()` functions to download the `df_school_all` dataframe from the url: `https://github.com/emoriebeck/psc290-data-FQ23/raw/main/05-assignments/05-school-visits`
 - Each row in `df_school_all` represents a high school (includes both public and private)
 - There are columns (e.g., `visit_by_100751`) indicating the number of times a university visited that high school
 - The variable `total_visits` identifies the number of visits the high school received from all (16) public research universities in this data collection sample
 - Use `names()` to identify the column names in the dataframe
 - Use `table()` to show the categorical values of the `school_type` column

```
load(url("https://github.com/emoriebeck/psc290-data-FQ23/raw/main/05-assignments/02-ps2/ps2_data.csv"))
names(df_school_all)
```

[1]	"state_code"	"school_type"	"ncessch"	"name"
[5]	"address"	"city"	"zip_code"	"pct_white"
[9]	"pct_black"	"pct_hispanic"	"pct_asian"	"pct_amerindian"
[13]	"pct_other"	"num_fr_lunch"	"total_students"	"num_took_math"
[17]	"num_prof_math"	"num_took_rla"	"num_prof_rla"	"med_inc"
[21]	"latitude"	"longitude"	"visits_by_196097"	"visits_by_186380"
[25]	"visits_by_215293"	"visits_by_201885"	"visits_by_181464"	"visits_by_139959"
[29]	"visits_by_218663"	"visits_by_100751"	"visits_by_199193"	"visits_by_110635"
[33]	"visits_by_110653"	"visits_by_126614"	"visits_by_155317"	"visits_by_106397"
[37]	"visits_by_149222"	"visits_by_166629"	"total_visits"	"inst_196097"
[41]	"inst_186380"	"inst_215293"	"inst_201885"	"inst_181464"
[45]	"inst_139959"	"inst_218663"	"inst_100751"	"inst_199193"
[49]	"inst_110635"	"inst_110653"	"inst_126614"	"inst_155317"

```
[53] "inst_106397"      "inst_149222"      "inst_166629"
```

```
table(df_school_all$school_type)
```

```
private  public
3822     17479
```

2. Use the tidyverse functions `arrange()` and `select()` to do the following:

- Sort `df_school_all` descending by `total_visits`
- Select the following variables: `name`, `state_code`, `city`, `school_type`, `total_visits`, `med_inc`, `pct_white`, `pct_black`, `pct_hispanic`, `pct_asian`, `pct_amerindian`
 - Note: You can do this in one step by wrapping the `select()` function around `arrange()`, or you can do this in two steps by creating an intermediate dataframe.

Print the first 10 rows of the final dataframe using `head()`, which represents the top 10 most visited schools by the 16 universities.

```
cols <- c("name", "state_code", "city", "school_type", "total_visits", "med_inc",
          "pct_white", "pct_black", "pct_hispanic", "pct_asian", "pct_amerindian")
```

```
df_school_all %>%
  arrange(desc(total_visits)) %>%
  select(all_of(cols)) %>%
  head(10)
```

```
# A tibble: 10 x 11
```

	name	state_code	city	school_type	total_visits	med_inc	pct_white	pct_black
	<chr>	<chr>	<chr>	<chr>	<int>	<dbl>	<dbl>	<dbl>
1	EPISCO~	VA	ALEX~	private	26	109558.	77.8	12.1
2	Lyons ~	IL	La G~	public	23	94306.	74.1	3.71
3	ALLEN ~	TX	ALLEN	public	23	100809	57.2	11.8
4	COPPEL~	TX	COPP~	public	23	123382.	49.9	4.97
5	FLOWER~	TX	FLOW~	public	22	157234.	74	3.06
6	NOLAN ~	TX	FORT~	private	21	39490.	55.8	3.47
7	FORT W~	TX	FORT~	private	20	89470.	4.09	2.82
8	LOVEJO~	TX	LUCAS	public	19	100809	81.9	1.91
9	STRAKE~	TX	HOUS~	private	18	29630.	56.7	7.76
10	TRINIT~	TX	ADDI~	private	18	77380	83.5	1.60

```
# i 3 more variables: pct_hispanic <dbl>, pct_asian <dbl>, pct_amerindian <dbl>
```

3. Building upon the previous question, print the following (select same variables as above):

- (A) Top 10 most visited public high schools in California
- (B) Top 10 most visited private high schools in California

```
df_school_all %>%
  arrange(desc(total_visits)) %>%
  select(all_of(cols)) %>%
  top_n(10, total_visits)
```

A tibble: 10 x 11

	name	state_code	city	school_type	total_visits	med_inc	pct_white	pct_black
	<chr>	<chr>	<chr>	<chr>	<int>	<dbl>	<dbl>	<dbl>
1	EPISCO~	VA	ALEX~	private	26	109558.	77.8	12.1
2	Lyons ~	IL	La G~	public	23	94306.	74.1	3.71
3	ALLEN ~	TX	ALLEN	public	23	100809	57.2	11.8
4	COPPEL~	TX	COPP~	public	23	123382.	49.9	4.97
5	FLOWER~	TX	FLOW~	public	22	157234.	74	3.06
6	NOLAN ~	TX	FORT~	private	21	39490.	55.8	3.47
7	FORT W~	TX	FORT~	private	20	89470.	4.09	2.82
8	LOVEJO~	TX	LUCAS	public	19	100809	81.9	1.91
9	STRAKE~	TX	HOUS~	private	18	29630.	56.7	7.76
10	TRINIT~	TX	ADDI~	private	18	77380	83.5	1.60

i 3 more variables: pct_hispanic <dbl>, pct_asian <dbl>, pct_amerindian <dbl>

```
df_school_all %>%
  arrange(desc(total_visits)) %>%
  select(all_of(cols)) %>%
  top_n(-10, total_visits)
```

A tibble: 15,405 x 11

	name	state_code	city	school_type	total_visits	med_inc	pct_white	pct_black
	<chr>	<chr>	<chr>	<chr>	<int>	<dbl>	<dbl>	<dbl>
1	Bethel~	AK	Beth~	public	0	76160	11.8	0.599
2	Ayagin~	AK	Kong~	public	0	76160	0	0
3	Kwigil~	AK	Kwig~	public	0	NA	0	0
4	Nelson~	AK	Toks~	public	0	57656.	0	0
5	Alakan~	AK	Alak~	public	0	37552.	2.52	0
6	Emmona~	AK	Emmo~	public	0	49219	0	0
7	Hooper~	AK	Hoop~	public	0	37500	0	0
8	Ignati~	AK	Moun~	public	0	35782.	0	0

```

 9 Pilot ~ AK          Pilo~ public          0  40573          0.559          0
10 Kotlik~ AK          Kotl~ public          0  34688.          0.538          0
# i 15,395 more rows
# i 3 more variables: pct_hispanic <dbl>, pct_asian <dbl>, pct_amerindian <dbl>

```

Render to pdf and submit problem set

Render to pdf by clicking the “Render” button near the top of your RStudio window (icon with blue yarn ball) or drop down and select “Knit to PDF”.

- Go to the Canvas and under the “Assignments”, submit to the Problem Set 1 Assignment.
- Submit both .qmd and .pdf files.
- Use this naming convention “lastname_firstname_ps#” for your .qmd and pdf files (e.g. beck-emorie-ps1.rmd & beck-emorie-ps1.pdf).